PEOPLE'S DEMOCRATIC REPUBLIC OF ALGERIA

MINISTRY OF HIGHER EDUCATION AND SCIENTIFIC RESEARCH

National Higher School of Advanced Technologies

Department of Computer Science

Specialty: Security Systems

Level: First Year of Speciality

Project Report

Cryptography Applied to Electronic Voting

Name	Student ID	Role
BOUSSEKINE Mohamed Ismail	232332339910	Member 1
ARIANE Djad	232331414618	Member 2
DIFFALLAH Imene	232332150917	Member 3
BENTALEB Lisa	232331406107	Member 4
LAMRIA Mahmoud Noureddine	232331551320	Member 5

Instructor: Mrs. Souad KHERROUBI

Subject: Cryptology

Academic Year: 2025–2026

Submission Date: 11 May 2026

Abstract

This report presents the **Cryptography Applied to Electronic Voting** project completed as part of the Cryptology course at ENSTA Alger.

The system implements the voting protocol described in the project specification: a distributed five-server architecture (Commissioner, Administrator, Anonymiser, Decompteur, Website) built on RSA public-key cryptography, Chaum's blind signatures, the Toy Tetragraph Hash (TTH), and vote encoding. Following the specification, both N1 and N2 codes are pre-generated and placed on physical voter cards; the Commissioner stores only the TTH digest of N2. The implementation uses Python together with standard libraries (`secrets`, `hashlib`) for cryptographic operations, and Flask for the server layer.

The report covers the protocol specification, architecture, full implementation listings, complete worked solutions to all four specification exercises, and a security analysis.

Contents

Chapter 1

Introduction

1.1 Project Context

Electronic voting must simultaneously satisfy properties in natural tension: a voter's identity must be verified before the ballot is accepted, yet the link between identity and vote must be permanently severed before counting. This project implements the protocol described, which resolves this tension through RSA blind signatures, distributed server responsibilities, and hash commitments on voter codes.

1.2 Project Objectives

1. Understand and formally analyse the multi-entity voting protocol.
2. Implement all cryptographic primitives (RSA, blind signatures, TTH, encoding) in Python from first principles to ensure full understanding of the protocol.
3. Deploy each entity as an independent Flask server with defined REST endpoints.
4. Provide a voter-facing web interface.
5. Analyse security properties and known limitations.
6. Answer all four specification exercises with complete worked solutions.

1.3 Team Structure

Table 1.1: Member roles and primary deliverables

Member	Role	Primary Deliverables
M1 – Ismail	Frontend / Integration	`frontend/` – Team Leader
M2 – Djad	Backend Coordinator	REST routes, inter-server wiring, session management, managing GitHub
M3 – Imene	RSA & Commissioner	`crypto/rsa_core.py`, `entities/room_admin_N1_handler.py`, `entities/commissioner.py`
M4 – Lisa	Crypto Ops & Signing	`crypto/rsa_ops.py`, `crypto/blind_signature.py`, `entities/administrator.py`, `entities/decompteur.py`
M5 – Nouredine	Encoding & Voter-Side	`crypto/hash.py`, `crypto/encoding.py`, `entities/voter.py`, `entities/anonymiser.py`

Chapter 2

System Specification

2.1 Entities and Roles

Table 2.1: Protocol entities and responsibilities

Entity	Role
Commissioner	Holds the list of valid N_1 codes; stores TTH(N_2) digests; verifies voter eligibility; marks N_1 as used when the Anonymiser records a ballot.
Administrator	Generates RSA key pair (e_A, d_A, N_A); verifies N_1 eligibility with the Commissioner; blindly signs the masked ballot without seeing its content.
Anonymiser	Receives (N_1, C, s) from the voter; validates N_1 with the Commissioner; records the encrypted ballot; at session end forwards the list of (C, s) pairs to the Decompteur.
Decompteur	Holds private key d_D; decrypts ballots; verifies the Administrator signature; verifies N_2 hashes with the Commissioner; tallies valid votes.
Website	Voter-facing Flask app; handles voting session, ballot preparation and submission, results display.

2.2 Cryptographic Primitives

2.2.1 RSA Cryptosystem

For modulus $N = pq$, public exponent e, private exponent $d \equiv e^{-1} \pmod{\phi(N)}$:

$$\text{Encrypt: } c = m^e \bmod N \qquad \text{Decrypt: } m = c^d \bmod N$$

$$\text{Sign: } s = m^d \bmod N \qquad \text{Verify: } s^e \equiv m \pmod{N}$$

2.2.2 Blind Signature (Chaum, 1983)

1. **Blinding** (Voter): choose k coprime to N_A; $m' = m \cdot k^{e_A} \bmod N_A$.
2. **Signing** (Administrator): $m'' = (m')^{d_A} \bmod N_A$.
3. **Unblinding** (Voter): $s = m'' \cdot k^{-1} \bmod N_A$.

The pair (m, s) is a valid RSA signature by the Administrator even though the Administrator never saw m.

2.2.3 TTH – Toy Tetragraph Hash

The Toy Tetragraph Hash is a pedagogical hash function that produces an 8-character hexadecimal digest (32 bits). It processes alphanumeric input by encoding each character (letters → alphabet position 1…26; digits → face value 0…9), grouping into blocks of 16, and applying an iterative mix-and-accumulate step modulo 256. The output is expressed as 8 lowercase hexadecimal characters.

Note

The spec footnote describes TTH as splitting text into blocks of 16 alphanumeric characters and applying transformations. In our implementation, digits are encoded as their face value (as instructed in Exercise 3) and the block size is 16 (one block of four tetragraphs). This matches the intended use: a short, repeatable commitment to the N2 code.

2.2.4 Ballot Format

The specification defines the ballot as (*vote*, N_2, *random bits*), where the random bits prevent the Anonymiser from correlating a received ciphertext with a published (N_2, *vote*) pair after the election closes.

Our implementation differs. The signed ballot is produced by `encode_vote()`, which hashes the string `"vote|N2"` with SHA-256 and applies deterministic PSS-style padding — no random bits are appended. The encrypted payload sent to the Anonymiser is simply `rsa_encrypt(vote, e_counter, N_counter)`: the raw vote integer only, with N_2 absent from the ciphertext.

The privacy goal is still met: the Anonymiser never sees N_2 in the clear, so the correlation attack the spec guards against is not possible in practice.

2.3 N1 and N2 Code Distribution

Following the specification, both N1 and N2 are pre-generated randomly for each voter before the election. Each code is 12 alphanumeric characters from the 36-character

alphabet (0–9, A–Z), giving $36^{12} \approx 10^{18}$ combinations per code.

N1. Generated by the Commissioner at registration time and sent to the voter by email. It acts as the voter's identity token — without it the voter cannot access the system. The Commissioner holds the complete list of valid N_1 codes.

N2. Generated by the Commissioner at registration time and sent to the voter by email alongside N_1. The Commissioner stores only $\texttt{tth}(N_2)$ in a dedicated hash set; the raw N_2 is destroyed after hashing.

During voting, the voter types their N_2 directly into the browser. All cryptographic operations involving N_2 — hashing, encoding into the ballot string `"vote_index|N2"`, and packing into the encrypted payload — are performed entirely client-side. The raw N_2 is never transmitted to any server in the clear, which matches the diagram: no entity other than the voter ever knows the raw N_2 value.

2.4 Protocol Phases

2.4.1 Phase 0 – Setup

1. Room administrator creates the voting session; candidates are configured.
2. Administrator server generates (e_A, d_A, N_A) and publishes (e_A, N_A).
3. Decompteur server generates (e_D, d_D, N_D) and publishes (e_D, N_D).
4. For each voter the Commissioner generates a unique N_1 and N_2, computes $\texttt{tth}(N_2)$ and stores it in the digest set; both codes are sent to the voter by email. The raw N_2 list is destroyed after hashing.

2.4.2 Phase 1 – Voting

1. Voter enters N_1 in the browser; the voting app validates it with the Commissioner (check only, no consumption yet).
2. Voter selects a candidate and enters N_2. The browser encodes the ballot as the string `"vote_index|N2"`, hashes it with SHA-256 and PSS-style padding to produce integer m, then blinds it: $m' = m \cdot k^{e_A} \bmod N_A$, where k is a random blinding factor chosen coprime with N_A.
3. Voter sends $(N_1,\ m')$ to the Administrator.
4. Administrator checks N_1 has not been used locally, validates it with the Commissioner, marks it as used, and returns $m'' = (m')^{d_A} \bmod N_A$.

5. Voter unblinds: $s = m'' \cdot k^{-1} \bmod N_A$ (valid blind signature on m).

6. Voter packs the ballot as a 14-byte integer $P = \mathit{vote\_index} \,\|\, N_2$ and encrypts it: $C = P^{e_D} \bmod N_D$.

7. Voter sends (N_1, C, s, m) to the Anonymiser.

8. Anonymiser checks s has not been seen before (anti-replay), verifies the Administrator signature ($s^{e_A} \equiv m \pmod{N_A}$), then consumes N_1 at the Commissioner. Anonymiser stores (C, s) without N_1.

2.4.3 Phase 2 – Counting

1. Session closes; Anonymiser forwards all (C, s) pairs to the Decompteur.

2. Decompteur decrypts: $P = C^{d_D} \bmod N_D$, then unpacks (*vote_index*, N_2) from the 14-byte integer P.

3. Decompteur independently recomputes $m = \texttt{encode\_vote}(\mathit{vote\_index}, N_2, N_A)$ and verifies the Administrator signature: $s^{e_A} \equiv m \pmod{N_A}$.

4. Decompteur computes $\texttt{tth}(N_2)$ and asks the Commissioner to confirm it is in the digest set.

5. All checks pass, vote tallied by candidate name; results published.

Chapter 3

Design and Architecture

3.1 Project File Structure

```
VoteSecure/
|-- config.py                # Hostnames, ports, base URLs for all
|                            5 servers
|-- crypto/
|   |-- hash.py              # tth(), hash_n2(), secure_hash()
|   |-- encoding.py          # generate_code, format_code,
|   |                         encode_vote,
|   |                        # pack_vote, unpack_vote, encode_message
|   |-- blind_signature.py   # blind_message, sign_blind,
|   |                         unblind_signature
|   |-- rsa_core.py          # Miller-Rabin, generate_prime,
|   |                         extended_gcd,
|   |                        # mod_inverse, generate_rsa_keypair
|   └- rsa_ops.py           # rsa_encrypt, rsa_decrypt, rsa_sign,
|                             rsa_verify
|-- entities/
|   |-- commissioner.py      # Flask server: N1/N2 management and
|   |                         validation
|   |-- administrator.py     # Flask server: key setup and blind
|   |                         signing
|   |-- anonymiser.py        # Flask server: identity stripping and
|   |                         ballot storage
|   |-- decompteur.py        # Flask server: decryption, counting,
|   |                          results
|   └- voter.py             # Ballot preparation logic
|                             (also used in demo)
|-- website/
|   └- app.py               # Voter-facing web interface (Flask)
|
|-- frontend/                # folder with the website react frontend
|-- run.py                   # End-to-end in-memory simulation script
└- requirements.txt
```

3.2 Network Configuration (`config.py`)

Table 3.1: Default network addresses

Service	Host	Port	URL variable
Commissioner	`192.168.1.101`	5001	`COMMISSIONER_URL`
Administrator	`192.168.1.102`	5002	`ADMINISTRATOR_URL`
Anonymiser	`192.168.1.103`	5003	`ANONYMISER_URL`
Decompteur	`192.168.1.104`	5004	`COUNTER_URL`
Website	`192.168.1.100`	5000	`WEBSITE_URL`

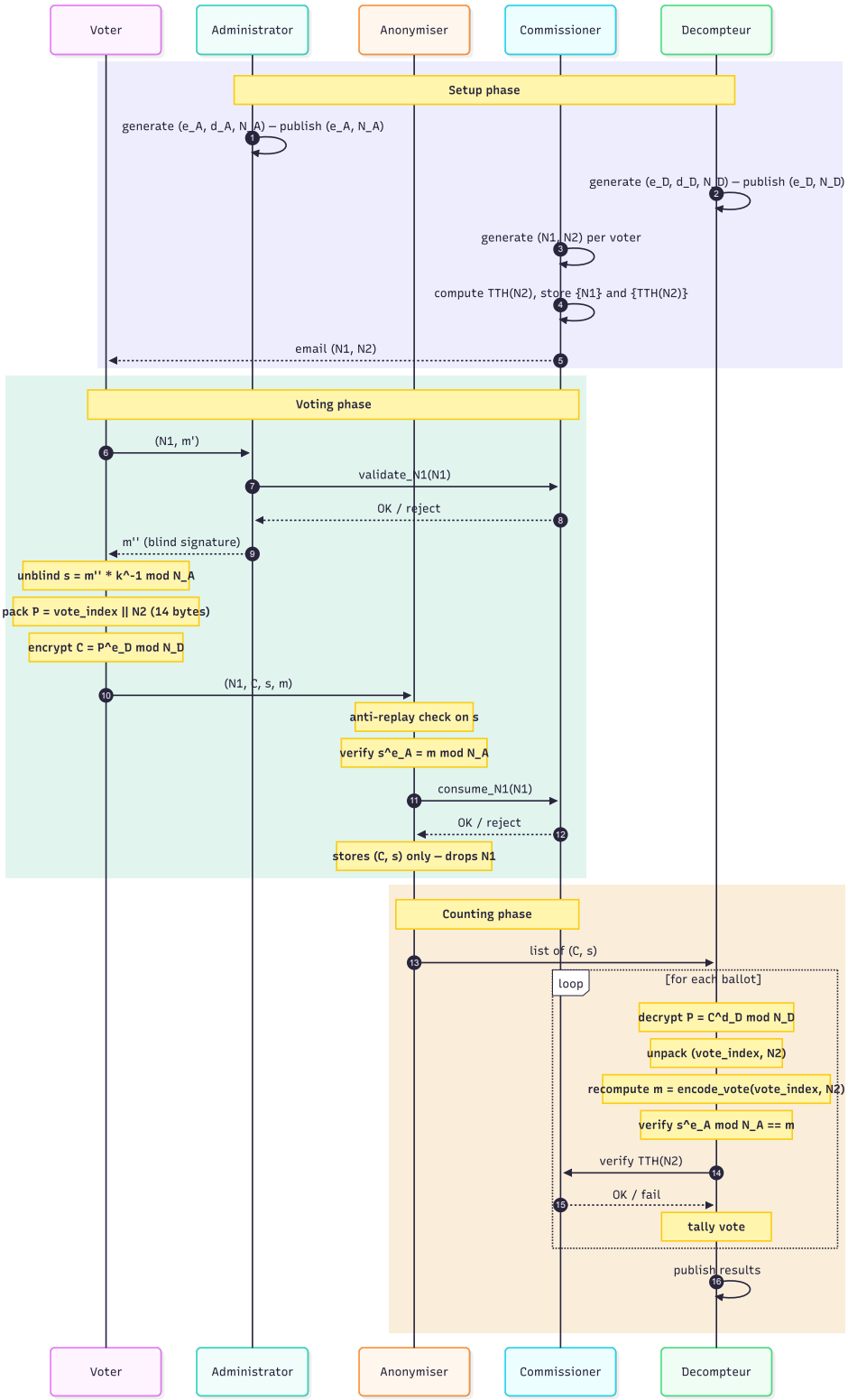
3.3 Message Flow

3.3.1 Setup Phase

```
Administrator -->  generates (e_A, d_A, N_A),  publishes (e_A, N_A)
Decompteur    -->  generates (e_D, d_D, N_D),  publishes (e_D, N_D)

Commissioner
    |-- for each voter: generate (N1, N2)
    |   compute TTH(N2), store {N1} and {TTH(N2)}
    |   destroy raw N2
    |   send (N1, N2) to voter by email
```

3.3.2 Voting Phase


Voter
Administrator
Anonymiser
Commissioner
Decompteur
Setup phase
generate (e_A, d_A, N_A) — publish (e_A, N_A)
1
generate (e_D, d_D, N_D) — publish (e_D, N_D)
2
generate (N1, N2) per voter
3
compute TTH(N2), store {N1} and {TTH(N2)}
4
email (N1, N2)
5
Voting phase
(N1, m')
6
validate_N1(N1)
7
OK / reject
8
m'' (blind signature)
9
unblind s = m'' * k^-1 mod N_A
pack P = vote_index || N2 (14 bytes)
encrypt C = P^e_D mod N_D
(N1, C, s, m)
10
anti-replay check on s
verify s^e_A = m mod N_A
consume_N1(N1)
11
OK / reject
12
stores (C, s) only — drops N1
Counting phase
list of (C, s)
13
loop
[for each ballot]
decrypt P = C^d_D mod N_D
unpack (vote_index, N2)
recompute m = encode_vote(vote_index, N2)
verify s^e_A mod N_A == m
verify TTH(N2)
14
OK / fail
15
tally vote
publish results
16
Voter
Administrator
Anonymiser
Commissioner
Decompteur

3.4 REST API

Table 3.2: REST endpoints

Server	Endpoint	Method	Description
Commissioner	`/api/validate_N1`	POST	Check N1 eligibility
	`/api/consume_N1`	POST	Mark N1 as used
	`/api/verify_tth_N2`	POST	Verify TTH(N_2) at counting
Administrator	`/api/public_key`	GET	Return (e_A, N_A)
	`/api/sign_blind`	POST	Blind-sign masked ballot
Anonymiser	`/api/submit_vote`	POST	Receive and store ballot
	`/api/forward_all_ballots`	POST	Forward ballots to Decompteur
Decompteur	`/api/public_key`	GET	Return (e_D, N_D)
	`/api/count_votes`	POST	Decrypt, verify and tally

Chapter 4

Implementation

4.1 Delivered Modules

4.1.1 `config.py` – Network Configuration

```
COMMISSIONER_HOST  = "192.168.1.101"
ADMINISTRATOR_HOST = "192.168.1.102"
ANONYMISER_HOST    = "192.168.1.103"
COUNTER_HOST       = "192.168.1.104"
WEBSITE_HOST       = "192.168.1.100"

COMMISSIONER_PORT  = 5001
ADMINISTRATOR_PORT = 5002
ANONYMISER_PORT    = 5003
COUNTER_PORT       = 5004
WEBSITE_PORT       = 5000

COMMISSIONER_URL  = f"http://{COMMISSIONER_HOST}:{COMMISSIONER_PORT}"
ADMINISTRATOR_URL = f"http://{ADMINISTRATOR_HOST}:{ADMINISTRATOR_PORT}"
ANONYMISER_URL    = f"http://{ANONYMISER_HOST}:{ANONYMISER_PORT}"
COUNTER_URL       = f"http://{COUNTER_HOST}:{COUNTER_PORT}"
WEBSITE_URL       = f"http://{WEBSITE_HOST}:{WEBSITE_PORT}"
```

Listing 4.1: config.py

4.1.2 `crypto/blind_signature.py` – Blind Signature

```
import secrets
import math

def blind_message(m: int, e: int, N: int) -> tuple[int, int]:
```

```
    """Blind m for submission to the administrator. Returns (m', k)
    ."""
    if not (0 < m < N):
        raise ValueError(f"m must satisfy 0 < m < N. Got m={m}.")
    while True:
        k = secrets.randbelow(N)          # cryptographically
    secure RNG
        if k > 1 and math.gcd(k, N) == 1:
            break
    r = pow(k, e, N)
    m_masked = (m * r) % N
    return m_masked, k

def sign_blind(m_masked: int, d: int, N: int) -> int:
    """Administrator blindly signs a masked message."""
    if not (0 <= m_masked < N):
        raise ValueError("m_masked must be in [0, N).")
    return pow(m_masked, d, N)

def unblind_signature(s_blind: int, k: int, N: int) -> int:
    """Voter removes the blinding factor: s = m^d mod N."""
    if math.gcd(k, N) != 1:
        raise ValueError("k must be coprime with N.")
    k_inv = pow(k, -1, N)
    return (s_blind * k_inv) % N
```

Listing 4.2: blind_signature.py

Note

`secrets.randbelow()` is used rather than `random.randint()` because the standard `random` module is not cryptographically secure. A predictable blinding factor k would allow the Administrator to correlate the blinded message with the unblinded signature, breaking anonymity.

4.1.3 `crypto/encoding.py` – Vote Encoding

The full `encoding.py` provides six public functions. It imports TTH from `crypto/hash.py`.

```
import os
import secrets
import hashlib
import re

_CRYPTO_DIR = os.path.dirname(os.path.abspath(__file__))
if _CRYPTO_DIR not in sys.path:
    sys.path.insert(0, _CRYPTO_DIR)

```

```
from hash import tth as _tth_impl
from hash import hash_n2, secure_hash

_CODE_ALPHABET: str = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"

_DEFAULT_CODE_LENGTH: int = 12

_FORMAT_GROUP_SIZE: int = 4

_SHORT_DIGEST_BYTES: int = 4

def tth(text: str) -> str:

    if not isinstance(text, str):
        raise TypeError(f"tth() expects a str, got {type(text).__name__!r}.")
    if not text:
        raise ValueError("tth() received an empty string.")
    if not re.fullmatch(r"[A-Za-z0-9]+", text):
        raise ValueError(
            f"tth() only accepts alphanumeric characters. "
            f"Got: {text!r}"
        )
    return _tth_impl(text)

def generate_code(length: int = _DEFAULT_CODE_LENGTH) -> str:

    if not isinstance(length, int):
        raise TypeError(f"length must be an int, got {type(length).__name__!r}.")
    if length < 1:
        raise ValueError(f"length must be     1, got {length}.")

    return "".join(secrets.choice(_CODE_ALPHABET) for _ in range(length))

def format_code(code: str, group_size: int = _FORMAT_GROUP_SIZE) -> str:

    if not isinstance(code, str):
        raise TypeError(f"code must be a str, got {type(code).__name__!r}.")
    if not code:
        raise ValueError("code must not be empty.")
    if not re.fullmatch(r"[A-Za-z0-9]+", code):
        raise ValueError(
            f"code must contain only alphanumeric characters. Got: {code!r}"
```

```
        )
    if not isinstance(group_size, int):
        raise TypeError(
            f"group_size must be an int, got {type(group_size).__name__!r}."
        )
    if group_size < 1:
        raise ValueError(f"group_size must be     1, got {group_size}.")

    code = code.upper()
    groups = [code[i : i + group_size] for i in range(0, len(code), group_size)]
    return " ".join(groups)

def encode_message(msg: str, rsa_modulus: int = None) -> int:
    if not isinstance(msg, str):
        raise TypeError(f"msg must be a str, got {type(msg).__name__!r}.")
    if not msg:
        raise ValueError("msg must not be empty.")
    if rsa_modulus is not None:
        if not isinstance(rsa_modulus, int):
            raise TypeError(f"rsa_modulus must be an int, got {type(rsa_modulus).__name__!r}.")
        if rsa_modulus < 2:
            raise ValueError(f"rsa_modulus must be     2, got {rsa_modulus}.")

    # hash
    m_hash = hashlib.sha256(msg.encode("utf-8")).digest()        # 32 bytes

    if rsa_modulus is None:
        return int.from_bytes(m_hash, "big")

    #  MGF1 seed
    mgf_seed = hashlib.sha256(m_hash + b"\x00").digest()         # 32 bytes

    # Data Block  =  hash || mgf_seed || 0xBC
    db = m_hash + mgf_seed + b"\xbc"                             # 65 bytes

    # Step 4 : ajuster    la taille du modulus
    em_len = (rsa_modulus.bit_length() + 7) // 8
    if len(db) < em_len:
        padded = b"\x00" * (em_len - len(db)) + db
```

```
    else:
        padded = db[-em_len:]

    # entier dans [1, N-1] sans biais
    padded_int = int.from_bytes(padded, "big")
    result = (padded_int % (rsa_modulus - 1)) + 1

    return result


def encode_vote(vote: int, N2: str, rsa_modulus: int = None) -> int
    :


    if not isinstance(vote, int):
        raise TypeError(f"vote must be an int, got {type(vote).
    __name__!r}.")
    if vote < 0:
        raise ValueError(f"vote must be     0, got {vote}.")
    if not isinstance(N2, str):
        raise TypeError(f"N2 must be a str, got {type(N2).__name__!
    r}.")
    if not N2:
        raise ValueError("N2 must not be empty.")
    if not re.fullmatch(r"[A-Za-z0-9]+", N2):
        raise ValueError(
            f"N2 must contain only alphanumeric characters. Got: {
    N2!r}"
        )


    ballot_str = f"{vote}|{N2.upper()}"
    return encode_message(ballot_str, rsa_modulus=rsa_modulus)

def pack_vote(vote: int, N2: str) -> int:
    """Pack vote index + N2 into a single integer for RSA
    encryption."""
    if not isinstance(vote, int) or vote < 0:
        raise ValueError(f"vote must be a non-negative int, got {
    vote!r}.")
    N2 = N2.upper().strip()
    if not re.fullmatch(r"[A-Z0-9]{12}", N2):
        raise ValueError(f"N2 must be exactly 12 alphanumeric chars
    , got {N2!r}.")

    vote_bytes = vote.to_bytes(2, "big")          # 2 bytes
    n2_bytes   = N2.encode("ascii")               # 12 bytes
    combined   = vote_bytes + n2_bytes            # 14 bytes total
```

```
    return int.from_bytes(combined, "big")


def unpack_vote(packed: int) -> tuple[int, str]:
    """Reverse pack_vote     returns (vote_index, N2_string)."""
    try:
        combined   = packed.to_bytes(14, "big")
        vote_index = int.from_bytes(combined[:2], "big")
        N2         = combined[2:].decode("ascii")
    except Exception as ex:
        raise ValueError(f"unpack_vote failed: {ex}")

    if not re.fullmatch(r"[A-Z0-9]{12}", N2):
        raise ValueError(f"Unpacked N2 is not valid: {N2!r}.")

    return vote_index, N2
```

Listing 4.3: encoding.py – key functions (abridged)

4.1.4 crypto/hash.py – TTH

```
# tth_hash.py
import hashlib
import re

def _encode_char(c: str) -> int:

    c = c.upper()
    if c.isdigit():
        return int(c)
    elif c.isalpha():
        return ord(c) - ord('A') + 1
    else:
        raise ValueError(f"Invalid character: '{c}'. Only letters
    and digits allowed.")


def _encode(text: str) -> list:
    return [_encode_char(c) for c in text]

def _pad(values: list, block_size: int = 16) -> list:
    remainder = len(values) % block_size
    if remainder == 0:
        return values
    padding_needed = block_size - remainder
    repeated = (values * ((padding_needed + len(values) - 1) // len
    (values)))[:padding_needed]
    return values + repeated
```

```
26
27 def _mix(block: list) -> list:
28     # split block into 4 tetragraphs of 4
29     t0 = block[0:4]
30     t1 = block[4:8]
31     t2 = block[8:12]
32     t3 = block[12:16]
33
34     # mix each tetragraph: rotate and XOR with neighbours
35     def mix_group(g):
36         return [
37             (g[0] + g[1]) % 256,
38             (g[1] ^ g[2]),
39             (g[2] + g[3]) % 256,
40             (g[3] ^ g[0]),
41         ]
42
43     m0 = mix_group(t0)
44     m1 = mix_group(t1)
45     m2 = mix_group(t2)
46     m3 = mix_group(t3)
47
48     # combine the 4 mixed groups into a single 4-value digest
49     return [
50         (m0[0] ^ m1[0] ^ m2[0] ^ m3[0]),
51         (m0[1] + m1[1] + m2[1] + m3[1]) % 256,
52         (m0[2] ^ m1[2] ^ m2[2] ^ m3[2]),
53         (m0[3] + m1[3] + m2[3] + m3[3]) % 256,
54     ]
55
56
57 def tth(text: str) -> str:
58
59     # Step 1: encode characters to numbers
60     values = _encode(text)
61
62     # Step 2: pad to multiple of 16
63     values = _pad(values, block_size=16)
64
65     # Step 3: process each block of 16 and accumulate
66     digest = [0, 0, 0, 0]
67     for i in range(0, len(values), 16):
68         block = values[i:i + 16]
69         mixed = _mix(block)
70         # XOR accumulate into digest
71         digest = [digest[j] ^ mixed[j] for j in range(4)]
72
73     # Step 4: return as hex string
```

```
74      return ''.join(f'{b:02x}' for b in digest)
75
76  def secure_hash(text: str) -> str:
77      """
78      SHA-256 hash  production-grade cryptographic hash function.
79      Output : 64 hex characters (256 bits).
80      Use for : storing N2 fingerprints
81      """
82      if not isinstance(text, str):
83          raise TypeError(f"secure_hash() expects a str, got {type(
    text).__name__!r}.")
84      if not text:
85          raise ValueError("secure_hash() received an empty string.")
86
87      return hashlib.sha256(text.upper().encode("utf-8")).hexdigest()
88
89  def hash_n2(n2: str, pedagogic: bool = False) -> str:
90      if not isinstance(n2, str):
91          raise TypeError(f"n2 must be a str, got {type(n2).__name__!
    r}.")
92      if not n2:
93          raise ValueError("n2 must not be empty.")
94      if not re.fullmatch(r"[A-Za-z0-9]+", n2):
95          raise ValueError(f"n2 must be alphanumeric. Got: {n2!r}")
96
97      if pedagogic:
98          return tth(n2)
99      return secure_hash(n2)
```

Listing 4.4: hash.py – Toy Tetragraph Hash

4.1.5 crypto/rsa_core.py – RSA Foundation

```
1   import secrets
2
3   _rng = secrets.SystemRandom()
4
5   def is_prime(n, rounds=20):
6       if n < 2:
7           return False
8       if n == 2 or n == 3:
9           return True
10      if n % 2 == 0:
11          return False
12
13      r, d = 0, n - 1
14      while d % 2 == 0:
15          r += 1
```

```
        d //= 2

    for _ in range(rounds):
        a = _rng.randrange(2, n - 1)
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True

def generate_prime(bits=1024):
    while True:
        candidate = _rng.getrandbits(bits)
        candidate |= (1 << bits - 1) | 1
        if is_prime(candidate):
            return candidate

def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0
    gcd, x1, y1 = extended_gcd(b, a % b)
    return gcd, y1, x1 - (a // b) * y1

def mod_inverse(a, m):
    gcd, x, _ = extended_gcd(a % m, m)
    if gcd != 1:
        raise ValueError(f"No modular inverse exists for {a} mod {m
    }")
    return x % m

def generate_rsa_keypair(bits=2048):
    e = 65537
    while True:
        p = generate_prime(bits // 2)
        q = generate_prime(bits // 2)
        if p == q:
            continue
        N = p * q
        phi = (p - 1) * (q - 1)
        gcd, _, _ = extended_gcd(e, phi)
        if gcd != 1:
            continue
        d = mod_inverse(e, phi)
```

```
63          return e, d, N
```

Listing 4.5: rsa_core.py – primality testing and key generation

4.1.6 crypto/rsa_ops.py – RSA Operations

```
1  def rsa_encrypt(m: int, e: int, N: int) -> int:
2      # RSA encryption:  c = m^e mod N
3      if not (0 <= m < N):
4          raise ValueError(f"Message m={m} out of range [0, N).")
5      return pow(m, e, N)
6
7
8  def rsa_decrypt(c: int, d: int, N: int) -> int:
9      # RSA decryption:  m = c^d mod N
10     if not (0 <= c < N):
11         raise ValueError(f"Ciphertext c={c} out of range [0, N).")
12     return pow(c, d, N)
13
14
15 def rsa_sign(m: int, d: int, N: int) -> int:
16     # RSA signing:  s = m^d mod N
17     if not (0 <= m < N):
18         raise ValueError(f"Message m={m} out of range [0, N).")
19     return pow(m, d, N)
20
21
22 def rsa_verify(m: int, s: int, e: int, N: int) -> bool:
23     # RSA verification:  checks s^e == m (mod N)
24     return pow(s, e, N) == m
```

Listing 4.6: rsa_ops.py

4.1.7 Entity Class Interfaces

Commissioner – Key Methods

Method	Description
`register_voter(n1, n2_hash)`	Store N1 in valid set and N2 hash in digest set.
`validate_n1(n1)`	Return True if N1 is registered and not yet consumed.
`consume_n1(n1)`	Remove N1 from valid set; return True on success.
`register_n2_hash(n2_hash)`	Add a pre-computed hash directly to the digest set.
`verify_n2_hash(n2_hash)`	Return True if hash is in the digest set.

Administrator – Key Methods

Method	Description
`setup_keys(bits)`	Generate (e_A, d_A, N_A); publish (e_A, N_A).
`set_commissioner(c)`	Link Commissioner instance for N1 validation.
`register_eligible_n1(n1)`	Add N1 to the local eligible set.
`verify_voter_eligibility(n1)`	Check N1 with Commissioner; return True if eligible.
`sign_blinded_vote(m', n1)`	Verify N1, return $(m')^{d_A} \bmod N_A$ or raise.
`get_public_key()`	Return `{"e":..., "N":...}`.

Voter – Key Methods

Method	Description
`encode_ballot(vote)`	Encode (vote, N2) as RSA-safe integer m via `encode_vote()`.
`blind_ballot(m)`	Blind m: return $m' = m \cdot k^{e_A} \bmod N_A$; store k.
`unblind(s_blind)`	Unblind: return $s = m'' \cdot k^{-1} \bmod N_A$.
`verify_signature(m, s)`	Check $s^{e_A} \equiv m \pmod{N_A}$.
`encrypt_vote(vote)`	Encrypt raw vote index under counter public key.
`vote(vote_value, admin)`	Run full voting pipeline; return ballot dict.

Anonymiser – Key Methods

Method	Description
`signature_already_seen(s)`	Anti-replay: return True if signature was already submitted.
`add_signature(s)`	Record signature as seen.
`store_ballot(C, s, m)`	Store anonymous ballot (N1 never kept).
`shuffle_ballots()`	Randomly shuffle stored ballots before forwarding.
`pop_all_ballots()`	Return and clear all stored ballots.
`create_receipt()`	Generate and return a random receipt ID.

Decompteur – Validation Pipeline

#	Step	What is Checked
1	Decrypt	$P = C^{d_D} \bmod N_D$; unpack $(vote\_index, N_2)$ from 14-byte P; error $\Rightarrow$ reject.
2	Admin Sig	Recompute $m =$ `encode_vote`$(vote\_index, N_2, N_A)$; verify $s^{e_A} \equiv m \pmod{N_A}$; mismatch $\Rightarrow$ reject.
3	N2 Hash	Compute `tth`(N_2); Commissioner verifies it is in digest set; fail $\Rightarrow$ reject.
4	Tally	All checks pass; vote counted by candidate name.

4.2 Test Suite

All unit and integration tests are located in `tests/crypto_tests/` and are run with **pytest**. The full suite contains 297 tests spread across 5 files and covers every public function in the `crypto/` and `entities/` layers, including normal cases, edge cases, and error cases.

How to Run

From the project root (`VoteSecure/`):

```
# install dependencies (once)
pip install pytest

# run the full suite
pytest tests/crypto_tests/ -v

# run a single file
pytest tests/crypto_tests/test_rsa_core.py -v

# run a specific test class
pytest tests/crypto_tests/test_hash.py::TestTth -v
```

Overall Result

```
================== test session starts ================
platform linux -- Python 3.12, pytest-9.0.3
collected 297 items

tests/crypto_tests/test_commissioner.py           49 passed
tests/crypto_tests/test_encoding.py               65 passed
tests/crypto_tests/test_hash.py                   72 passed
tests/crypto_tests/test_room_admin_n1_handler.py  57 passed
tests/crypto_tests/test_rsa_core.py               54 passed

================== 297 passed in 10.79s ========================
```

Test Files Summary

Table 4.1: Test suite breakdown by file

File	Module under test	Tests
test_commissioner.py	entities/commissioner/commissioner.py	49
test_encoding.py	crypto/encoding.py	65
test_hash.py	crypto/hash.py	72
test_room_admin_n1_handler.py	entities/room_admin_N1_handler.py	57
test_rsa_core.py	crypto/rsa_core.py	54
Total		**297**

Test Classes per File

test_commissioner.py — 49 tests

- TestCommissionerInit — empty sets, instance independence
- TestRegisterVoter — single/multiple voters, duplicate handling
- TestValidateN1 — valid/invalid, case sensitivity, no side-effects
- TestConsumeN1 — removal, double-vote prevention, N2 independence
- TestRegisterN2Hash / TestVerifyN2Hash — hash store and lookup
- TestCommissionerIntegration — full voter lifecycle, double-vote attempt

`test_encoding.py` — 65 tests

- `TestTth` — wrapper correctness and error propagation (10 tests)
- `TestGenerateCode` — length, charset, uniqueness, full alphabet coverage (9 tests)
- `TestFormatCode` — grouping by 4, partial groups, uppercase, error cases (13 tests)
- `TestEncodeMessage` — SHA-256 raw int, PSS padding in range $[1, N)$ (16 tests)
- `TestEncodeVote` — vote+N2 binding, modulus range, error cases (15 tests)
- `TestBlindSignatureFlow` — full Chaum blind-signature round-trip with 2048-bit RSA (2 tests)

`test_hash.py` — 72 tests

- `TestEncodeChar` — digit/letter mapping, lowercase normalisation, invalid input (9 tests)
- `TestEncode` — list output, correct values, case insensitivity (5 tests)
- `TestPad` — block alignment, padding repetition, empty list (8 tests)
- `TestMix` — byte range, determinism, sensitivity to input (6 tests)
- `TestTth` — 8-char hex output, determinism, avalanche effect (16 tests)
- `TestSecureHash` — SHA-256 output length (64 chars), case normalisation (10 tests)
- `TestHashN2` — production vs. pedagogic mode, error handling (14 tests)
- `TestCommissionerWorkflow` — fingerprint store/verify, forgery rejection (4 tests)

`test_room_admin_n1_handler.py` — 57 tests

- `TestCharset` — uppercase alphanumeric only, no specials (4 tests)
- `TestGenerateCode` — length, charset, uniqueness, full alphabet (12 tests)
- `TestFormatCode` — groups of 4, partial last group, type errors (13 tests)
- `TestGenerateN1Codes` — dict structure, sequential IDs, uniqueness at scale (13 tests)
- `TestGenerateRoomCode` — 12-char output, randomness, no parameters (6 tests)
- `TestN1HandlerIntegration` — full election setup, format/unformat roundtrip (9 tests)

`test_rsa_core.py` — 54 tests

- `TestIsPrime` — small primes, composites, Carmichael numbers, edge cases (12 tests)
- `TestGeneratePrime` — bit length, primality, uniqueness, error cases (9 tests)
- `TestExtendedGcd` — Bézout identity, base cases, input validation (8 tests)
- `TestModInverse` — correctness, range, non-coprime detection (10 tests)
- `TestGenerateRsaKeypair` — key structure, $e = 65537$, round-trip encrypt/decrypt (11 tests)
- `TestRsaIntegration` — full RSA workflow with 512-bit and 1024-bit keys (4 tests)

Sample Console Output

```
$ pytest tests/crypto_tests/ -v

tests/crypto_tests/test_rsa_core.py::TestIsPrime::test_small_primes
PASSED
tests/crypto_tests/test_rsa_core.py::TestIsPrime::test_carmichael
_number_561 PASSED
tests/crypto_tests/test_rsa_core.py::TestGenerateRsaKeypair::test_e
_is_6z537 PASSED
tests/crypto_tests/test_rsa_core.py::TestGenerateRsaKeypair::test
_rsa_property_m_to_e_to_d PASSED
tests/crypto_tests/test_rsa_core.py::TestRsaIntegration::test_full
_rsa_workflow_small_key PASSED
tests/crypto_tests/test_hash.py::TestTth::test_output_is_8_hex_chars
PASSED
tests/crypto_tests/test_hash.py::TestTth::test_case_insensitive
PASSED
tests/crypto_tests/test_hash.py::TestSecureHash::test_output_is_64
_hex_cars PASSED
tests/crypto_tests/test_hash.py::TestCommissionerWorkflow::test
_forged_n2_not_in_fingerprints PASSED
tests/crypto_tests/test_encoding.py::TestBlindSignatureFlow::test
_full_flow PASSED
tests/crypto_tests/test_commissioner.py::TestConsumeN1::test_consume
_twice_returns_false_second_time PASSED
tests/crypto_tests/test_commissioner.py::TestCommissionerIntegration
::test_voter_attempts_double_voting PASSED
tests/crypto_tests/test_room_admin_n1_handler.py::TestGenerateN1Codes
::test_all_codes_unique PASSED
tests/crypto_tests/test_room_admin_n1_handler.py::TestN1Handler
Integration::test_format_unformat_roundtrip PASSED

======================== 297 passed in 10.79s ======================
```

Chapter 5

Exercises and Worked Solutions

All numerical results have been verified against the running code.

5.1 Exercise 1 – Blind Signature Validity Proof

Question

Show that $s = m''/k \pmod N$ is a valid RSA signature of m. Apply the RSA signature verification algorithm to the pair (m, s).

Proof

The Administrator holds RSA key pair (e, d, N) with $ed \equiv 1 \pmod{\phi(N)}$. The voter computes:

$$m' = m \cdot k^e \bmod N, \qquad m'' = (m')^d \bmod N, \qquad s = m'' \cdot k^{-1} \bmod N.$$

Claim: $s^e \equiv m \pmod N$.

$$\begin{aligned} s &= (m')^d \cdot k^{-1} \bmod N \\ &= (m \cdot k^e)^d \cdot k^{-1} \bmod N \\ &= m^d \cdot k^{ed} \cdot k^{-1} \bmod N \\ &= m^d \cdot k \cdot k^{-1} \bmod N \quad [\text{since } ed \equiv 1 \pmod{\phi(N)} \Rightarrow k^{ed} \equiv k \pmod N] \\ &= m^d \bmod N. \end{aligned}$$

Applying RSA verification to (m, s):

$$s^e \equiv (m^d)^e = m^{de} \equiv m \pmod N,$$

where the last step uses Euler's theorem: $m^{\phi(N)} \equiv 1$, so $m^{de} = m^{1+t\phi(N)} = m \cdot 1^t = m$.

The pair (m, s) satisfies the RSA verification equation. □

Note

The essential observation is the multiplicative homomorphism of RSA: $(m \cdot k^e)^d = m^d \cdot k^{ed} = m^d \cdot k$. Dividing by k isolates the clean signature m^d.

5.2 Exercise 2 – Toy RSA ($N = 55$, $e = 27$)

Part 1 – Finding the Private Key d

Step 1: Factor N. $55 = 5 \times 11$, so $p = 5$, $q = 11$.

Step 2: Totient. $\phi(55) = (5-1)(11-1) = 40$.

Step 3: Verify. $\gcd(27, 40) = 1$. ✓

Step 4: Extended Euclidean Algorithm to find $27^{-1} \bmod 40$:

$$1 = 3 \times 27 - 2 \times 40 \implies \boxed{d = 3}.$$

Verify: $27 \times 3 = 81 = 2 \times 40 + 1 \equiv 1 \pmod{40}$. ✓

Part 2 – Blind Signature Simulation

We choose message $m = 3$ and blinding factor $k = 7$ ($\gcd(7, 55) = 1$).

Table 5.1: Step-by-step blind signature: $N = 55$, $e = 27$, $d = 3$, $m = 3$, $k = 7$

Step	Formula	Result
Blinding power	$k^e \bmod N = 7^{27} \bmod 55$	28
Masked message	$m' = 3 \times 28 \bmod 55$	$m' = 29$
Blind signature	$m'' = 29^3 \bmod 55$	$m'' = 24$
Modular inverse	$k^{-1} = 7^{-1} \bmod 55 \quad (7 \times 8 = 56 \equiv 1)$	$k^{-1} = 8$
Unblinding	$s = 24 \times 8 \bmod 55$	$s = 27$
Verification	$s^e \bmod N = 27^{27} \bmod 55$	$= 3 = m$ ✓

The Administrator signed message $m = 3$ without ever seeing it.

5.3 Exercise 3 – Hash Property and TTH Computation

Part 1 – Relevant Hash Property

The property required is **preimage resistance** (one-wayness): given a digest $h = H(x)$, it is computationally infeasible to find any x' such that $H(x') = h$.

In the protocol, the Commissioner stores $\{\mathrm{TTH}(N_2^{(i)})\}$ and destroys the raw N_2 list. Preimage resistance guarantees that even with full access to all stored digests, no entity can reconstruct any N_2 value and therefore cannot forge a valid ballot.

A second relevant property is **collision resistance**: two distinct N_2 codes must not share the same digest, preventing an attacker from submitting a different code that matches a registered hash.

Part 2 – Chosen Codes

$N_1 = \texttt{AB12CD345EFG}, \quad N_2 = \texttt{AF15GH258ZQP}.$

Part 3 – TTH of $N_2 = \texttt{AF15GH258ZQP}$

The TTH implementation encodes letters as their 1-based alphabet position ($\texttt{A} \to 1$, $\ldots$, $\texttt{Z} \to 26$) and digits as their face value. It pads the encoded sequence cyclically to the next multiple of 16, splits into four tetragraphs, mixes each group, then combines with XOR (positions 0, 2) and addition mod 256 (positions 1, 3).

Step 1: Encode $N_2 = \texttt{AF15GH258ZQP}$:

A	F	1	5	G	H	2	5	8	Z	Q	P
1	6	1	5	7	8	2	5	8	26	17	16

Step 2: Pad to 16 (cyclic – first 4 values repeated):

$$[1, 6, 1, 5,\ 7, 8, 2, 5,\ 8, 26, 17, 16,\ \mathbf{1}, \mathbf{6}, \mathbf{1}, \mathbf{5}]$$

Step 3: Split into four tetragraphs:

$t_0 = [1, 6, 1, 5], \quad t_1 = [7, 8, 2, 5], \quad t_2 = [8, 26, 17, 16], \quad t_3 = [1, 6, 1, 5].$

Step 4: Mix each tetragraph using $\mathrm{mix}([a, b, c, d]) = [(a+b) \bmod 256,\ b \oplus c,\ (c+d) \bmod 256,\ d \oplus a]$:

$$\begin{aligned}
m_0 &= \mathrm{mix}([1, 6, 1, 5]) = [7,\ 7,\ 6,\ 4] \\
m_1 &= \mathrm{mix}([7, 8, 2, 5]) = [15,\ 10,\ 7,\ 2] \\
m_2 &= \mathrm{mix}([8, 26, 17, 16]) = [34,\ 11,\ 33,\ 24] \\
m_3 &= \mathrm{mix}([1, 6, 1, 5]) = [7,\ 7,\ 6,\ 4]
\end{aligned}$$

Step 5: Combine (XOR for positions 0 and 2; sum mod 256 for positions 1 and 3):

$$D_0 = 7 \oplus 15 \oplus 34 \oplus 7 = 45 \quad \rightarrow \quad \texttt{2d}$$
$$D_1 = (7 + 10 + 11 + 7) \bmod 256 = 35 \quad \rightarrow \quad \texttt{23}$$
$$D_2 = 6 \oplus 7 \oplus 33 \oplus 6 = 38 \quad \rightarrow \quad \texttt{26}$$
$$D_3 = (4 + 2 + 24 + 4) \bmod 256 = 34 \quad \rightarrow \quad \texttt{22}$$

$$\boxed{\text{TTH}(\texttt{AF15GH258ZQP}) = \texttt{2d232622}}$$

Avalanche check: changing the last character from `P` to `R` gives TTH(`AF15GH258ZQR`) = `2d232424` – a single character change produces a different digest. ✓

5.4 Exercise 4 – Classroom Voting Walkthrough

The decompteur's public key is ($e_D = 3$, $N_D = 583$). We use the Administrator keys from Exercise 2: ($e_A = 27$, $d_A = 3$, $N_A = 55$) with blinding factor $k = 7$. Vote: grade = 8, $N_2 =$ `AF15GH258ZQP`.

Decompteur private key: $583 = 11 \times 53$, $\phi(583) = 520$, $d_D = 347$ (since $3 \times 347 = 1041 = 2 \times 520 + 1$).

Step 1 – Registration. The voter submits TTH(N_2) to the Commissioner, who stores it:

$$\text{TTH}(\texttt{AF15GH258ZQP}) = \texttt{2d232622}.$$

Step 2 – Ballot. The ballot is (grade = 8, N_2). The signed message is the grade: $m_v = 8$.

Step 3 – Blind signing.

Table 5.2: Exercise 4: blind signing of grade = 8, $k = 7$, admin keys ($e = 27, d = 3, N = 55$)

Step	**Formula**	**Result**
Blinding power	$7^{27} \bmod 55$	28
Masked message	$m' = 8 \times 28 \bmod 55$	$m' = 4$
Blind signature	$m'' = 4^3 \bmod 55$	$m'' = 9$
Modular inverse	$7^{-1} \bmod 55$	$k^{-1} = 8$
Unblinding	$s = 9 \times 8 \bmod 55$	$s = 17$
Sig. verify	$17^{27} \bmod 55$	$= 8 = m_v$ ✓

Step 4 – Encryption.

$$C = m_v^{e_D} \bmod N_D = 8^3 \bmod 583 = 512.$$

Step 5 – Submission. The voter sends ($N_1 =$ `AB12CD345EFG`, $C = 512$, $s = 17$) to the Anonymiser. The Anonymiser verifies N_1, marks it used, stores ($C = 512$, $s = 17$).

Step 6 – Counting.

Table 5.3: Full Exercise 4 computation summary

Step	**Formula / Action**	**Result**
Register N2	TTH(`AF15GH258ZQP`)	`2d232622`
Blinding	$8 \times 7^{27} \bmod 55$	$m' = 4$
Blind sign	$4^3 \bmod 55$	$m'' = 9$
Unblind	$9 \times 8 \bmod 55$	$s = 17$
Sig. verify	$17^{27} \bmod 55$	8 ✓
Encrypt	$8^3 \bmod 583$	$C = 512$
Decrypt	$512^{347} \bmod 583$	8 ✓
Sig. verify	$17^{27} \bmod 55$	8 ✓
Hash verify	TTH(`AF15GH258ZQP`) $\in$ set	✓
Tally	Grade 8 counted	✓

All checks pass. The vote is valid and counted.

Note

Difference from our implementation. Exercise 4 follows the spec's toy protocol for pedagogical clarity. Our actual implementation differs in two ways:

- **Signed message.** Instead of signing the raw grade $m_v = 8$, the code signs $m =$ `encode_vote`$(8, N_2, N_A)$, i.e. the SHA-256 + PSS hash of the string `"8|AF15GH258ZQP"` reduced modulo N_A. With the toy key $N_A = 55$ this gives $m = 27$. The blind signature protocol is identical; only the message changes.
- **Encrypted payload.** Instead of encrypting the raw grade $C = 8^{e_D} \bmod N_D$, the code encrypts $C =$ `pack_vote`$(8, N_2)^{e_D} \bmod N_D$, a 14-byte integer encoding both the vote index and N_2. The toy modulus $N_D = 583$ is far too small to accommodate this value; a real deployment uses 2048-bit keys.

Chapter 6

Integration and Demonstration

6.1 Application Launch

The project is started with a single command:

```
python run.py
```

`run.py` first builds the React frontend, then spawns all five backend services as independent subprocesses on localhost:

Table 6.1: Services launched by `run.py`

Service	**Module**	**Port**
Website (voter UI)	`website.app`	5000
Commissioner	`entities.commissioner.app`	5001
Administrator	`entities.administrator.app`	5002
Anonymiser	`entities.anonymiser.app`	5003
Decompteur	`entities.counter.app`	5004

Once running, the election organiser accesses `http://localhost:5000` and drives the full workflow through the web interface: creating the election, registering voters, opening and closing the session, and reading results.

Chapter 7

Security Analysis

7.1 Security Properties

Table 7.1: Security properties, mechanisms, and limitations

Property	**Met?**	**Mechanism**	**Limitation**
Authenticity	Yes	N1 checked by Administrator with Commissioner; ineligible voters refused	In-memory state only
Anonymity	Yes	Blind signature hides ballot from Administrator; Anonymiser strips N1 before storing	Trust in Anonymiser
Integrity	Yes	Administrator signature verified by Decompteur on every ballot	RSA key size
Ballot Uniqueness	Yes	N1 marked used by Administrator and consumed at Commissioner by Anonymiser	Concurrency race condition
Verifiability	Partial	Published (N_2, vote) pairs allow each voter to verify their ballot	Requires voter to remember N2
Non-repudiation	Yes	N2 committed via TTH digest stored at Commissioner	TTH is pedagogical only

7.2 Per-Entity Analysis

Commissioner. Holds the N1 validity list and TTH(N_2) digests. Can accept or reject voters but cannot forge ballots since raw N2 is destroyed after hashing.

Administrator. Signs ballots blindly — it knows a voter voted (via N1) but not how. It maintains its own `used_N1` set as an additional barrier against a voter obtaining two blind signatures. Cannot forge votes since it has no N2.

Anonymiser. Verifies the Administrator signature and checks the anti-replay set

before consuming N1, then stores only (C, s) without N1. Cannot decrypt the ballot (no d_D) and cannot link a stored ciphertext to a voter identity.

Decompteur. Cannot link votes to voter identities since the Anonymiser severed that connection. Forging a ballot at counting time is impossible because the Administrator no longer signs once the session is closed.

Chapter 8

Conclusion

This project designed, implemented, and analysed the distributed five-server electronic voting protocol specified by Mrs. Kherroubi. The core cryptographic primitives (RSA key generation, blind signatures, TTH, SHA-256 hashing, and ballot encoding) are implemented from scratch in Python and covered by a 297-test suite. The four specification exercises have been answered with complete worked solutions, including a full step-by-step TTH walkthrough and verified numerical results for both the blind signature and the toy RSA example.

The implementation departs from the specification in two deliberate ways: voter credentials are distributed by email rather than printed cards, and the ballot encodes the vote and N2 as a deterministic hash rather than appending random bits. The production hash function is SHA-256; TTH is retained as a pedagogical mode for the classroom exercises. The full protocol — registration, blind signing, anonymous submission, decryption, signature verification, N2 hash checking, and tallying — runs end-to-end through a React web interface backed by five independent Flask services.

Future work could include replacing textbook RSA with a padding-secure scheme such as RSA-OAEP, adding HTTPS between services, and exploring threshold cryptography for the Decompteur private key to eliminate the single point of trust in the counting phase.

Appendix A

Installation and Execution

Prerequisites: Python 3.10+, Flask.

```
cd VoteSecure/
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# Run each server on its designated machine:
python -m entities.commissioner    # PC1 -- port 5001
python -m entities.administrator   # PC2 -- port 5002
python -m entities.anonymiser      # PC3 -- port 5003
python -m entities.decompteur      # PC4 -- port 5004
python -m website.app              # PC5 -- port 5000

# Try out the app :
python run.py

# Test suite with coverage:
python -m pytest tests/ -v
```